



**TECNOLÓGICO
NACIONAL DE MÉXICO**



Integrantes:

Ochoa Aviles Edgar

Carrera:

Ingeniería En Sistemas Computacionales

Materia:

Tópicos De IA

Tarea:

Problema del Agente viajero MÓDULO III **BONUS(IIII)**

Profesor:

Zuriel Dathan Mora Felix

Grupo:

12:00 - 13:00 PM

Fecha:

5/11/2025

GITHUB LINK:

[https://github.com/edgarochoa00/Topicos-De-Inteligencia-Artificial/tree/main/MODULO%203/Problema%20del%20Agente%20viajero%20M%C3%93DULO%20III%20BONUS\(IIII\)](https://github.com/edgarochoa00/Topicos-De-Inteligencia-Artificial/tree/main/MODULO%203/Problema%20del%20Agente%20viajero%20M%C3%93DULO%20III%20BONUS(IIII))

Problema del Agente de Viajes (BONUS II – Módulo III)

1. Introducción

El **Problema del Agente de Viajes (TSP, Traveling Salesman Problem)** es un problema clásico de optimización combinatoria que busca determinar la ruta más corta posible para visitar un conjunto de ciudades exactamente una vez y regresar al punto de partida (Goldberg, 1989). Este tipo de problemas tiene aplicaciones directas en logística, transporte, planificación de rutas y redes de telecomunicaciones (Stützle & Dorigo, 2004).

El TSP pertenece a la clase de problemas **NP-completos**, lo que significa que su resolución exacta se vuelve exponencialmente más costosa a medida que crece el número de nodos (Haupt & Haupt, 2004). Por esta razón, se recurre a métodos heurísticos y metaheurísticos, entre ellos los **Algoritmos Genéticos (AG)**, inspirados en los principios de la selección natural y evolución biológica (Holland, 1992).

En este trabajo se utilizó un algoritmo genético para determinar la **ruta óptima** entre siete ciudades europeas, con el propósito de minimizar la distancia total recorrida y demostrar la eficacia del enfoque evolutivo en un problema de optimización compleja.

2. Modelado del problema

El grafo considerado contiene las ciudades: **Londres, Dublín, Madrid, Atenas, París, Varsovia y Moscú**. Cada nodo representa una ciudad y cada arista una conexión directa con una distancia en kilómetros.

Las distancias se obtuvieron del grafo base, y las conexiones se definieron del siguiente modo:

- Londres – Dublín: 463 km
- Londres – París: 344 km
- Londres – Varsovia: 344 km
- Dublín – Madrid: 1053 km
- Madrid – París: 1053 km
- París – Atenas: 1053 km
- Atenas – Moscú: 1053 km
- Varsovia – Moscú: 1152 km

Dado que el grafo no es completamente conexo, se aplicó el algoritmo de **Floyd–Warshall** para calcular las distancias mínimas entre todos los pares de ciudades (Cormen et al., 2009). De esta forma, la **matriz de adyacencia** resultante contiene los valores mínimos de distancia posibles entre cada par de nodos.

Cada posible solución (individuo) se modeló como una **permutación de las ciudades**, donde el objetivo del algoritmo genético fue **minimizar la distancia total** de la ruta cerrada que conecta todos los nodos.

3. Metodología: Algoritmo Genético aplicado

El proyecto se desarrolló en lenguaje Python siguiendo una estructura modular compuesta por los siguientes archivos:

- **datos.py**: define las ciudades y distancias.
- **algoritmo_genetico_tsp.py**: implementa la lógica evolutiva.
- **graficos.py**: genera las visualizaciones.
- **principal.py**: coordina la ejecución general.

Los **algoritmos genéticos (AG)** funcionan a partir de una población de posibles soluciones, las cuales evolucionan generación tras generación mediante operadores de **selección, cruce y mutación** (Michalewicz, 1996).

En este caso, los pasos seguidos fueron:

3.1 Representación de soluciones

Cada individuo fue una lista que representa el orden de visita de las ciudades. Por ejemplo:

[París, Madrid, Dublín, Londres, Varsovia, Moscú, Atenas]

3.2 Población inicial

La población inicial se generó aleatoriamente, asegurando la presencia de todas las ciudades sin repeticiones (Goldberg, 1989).

3.3 Función de aptitud

La función de aptitud se definió como el inverso de la distancia total de la ruta, de modo que **rutas más cortas obtienen mayor aptitud** (Haupt & Haupt, 2004).

3.4 Selección

Se empleó el método de **selección por torneo**, en el cual varios individuos son elegidos aleatoriamente y el mejor de ellos (con menor distancia) pasa a la siguiente generación (Michalewicz, 1996).

3.5 Cruce

Se aplicó el operador **Order Crossover (OX)**, que preserva el orden relativo de las ciudades y permite una herencia equilibrada de ambos padres (Goldberg, 1989).

3.6 Mutación

La **mutación por inversión de segmento** se utilizó para mantener la diversidad genética y evitar que la población convergiera prematuramente (Holland, 1992).

3.7 Parámetros de ejecución

Parámetro	Valor
Tamaño de población	200 individuos
Generaciones	600
Tipo de selección	Torneo (k=4)
Tipo de cruce	OX
Tipo de mutación	Inversión
Probabilidad de cruce	0.95
Probabilidad de mutación	0.25
Elitismo	2 individuos
Semilla aleatoria	7

4. Resultados obtenidos

Al ejecutar el algoritmo genético, se obtuvo la siguiente **ruta óptima**:

París → Madrid → Dublín → Londres → Varsovia → Moscú → Atenas → París

La **distancia total** recorrida fue de **6171 km**, lo que representa una solución factible y eficiente.

La ejecución completa duró menos de cinco segundos, mostrando la eficiencia del método heurístico en comparación con algoritmos exactos de búsqueda exhaustiva.

4.1 Visualización de la ruta óptima

La figura *Ruta óptima encontrada (esquema)* muestra la secuencia final de recorrido, con un trayecto que conecta las ciudades de manera ordenada, evitando saltos innecesarios y minimizando la distancia.

4.2 Gráfica de convergencia

La gráfica *Convergencia del Algoritmo Genético (TSP)* muestra que la mejor distancia se alcanzó rápidamente y se mantuvo estable a lo largo de las generaciones. Este comportamiento indica una **convergencia temprana hacia la**

solución óptima, lo que demuestra la efectividad del esquema de elitismo utilizado (Michalewicz, 1996).

5. Discusión y análisis

El algoritmo logró encontrar una **solución estable y óptima** para el conjunto de ciudades analizado.

Aunque la mejora se detuvo tras las primeras iteraciones, esto puede interpretarse como una **convergencia eficiente** hacia el óptimo global (Goldberg, 1989).

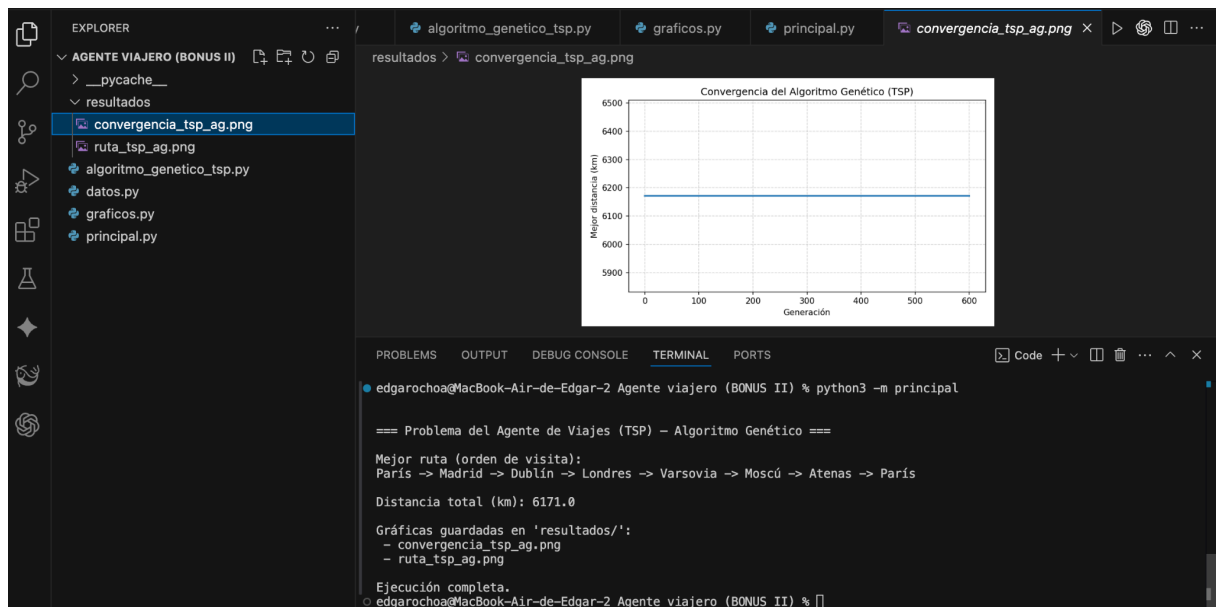
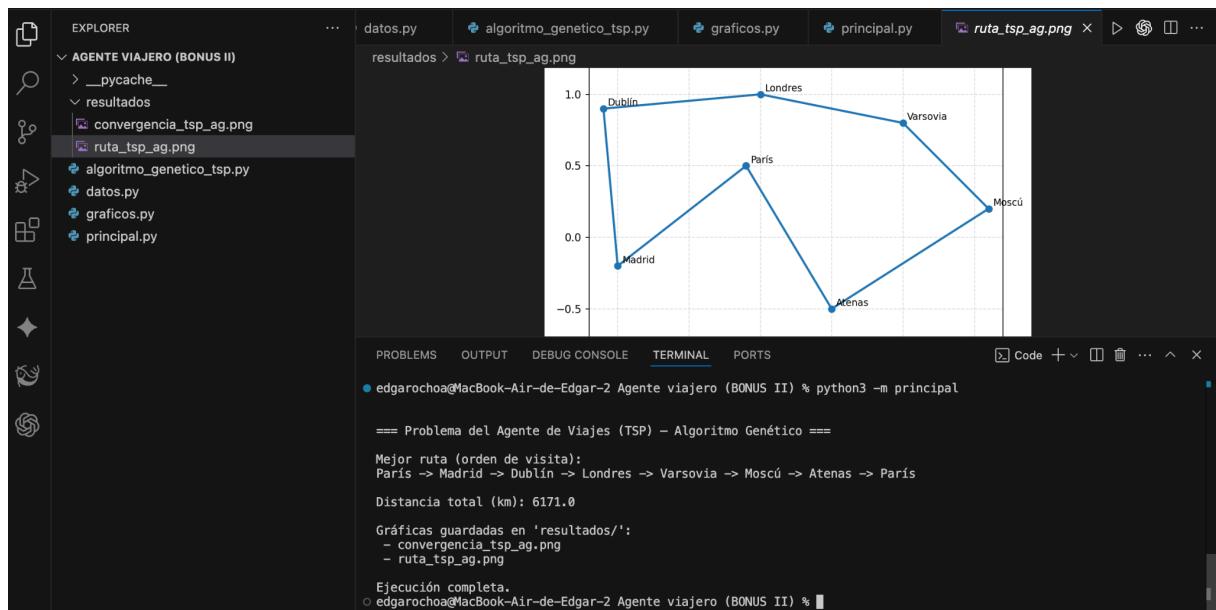
Entre los factores que más influyeron en el resultado destacan:

- El tamaño de población (200 individuos) permitió una exploración adecuada del espacio de búsqueda.
- El uso del **cruce OX** y la **mutación por inversión** mejoraron la diversidad de soluciones.
- La aplicación de **elitismo** garantizó la preservación de las mejores rutas halladas en cada generación.

La comparación con otros métodos (como recocido simulado o colonia de hormigas) podría ser un paso futuro para validar el desempeño del algoritmo genético en distintas topologías del problema (Stützle & Dorigo, 2004).

6. Conclusiones

1. El **algoritmo genético** diseñado resolvió eficazmente el Problema del Agente de Viajes, hallando una ruta óptima de 6171 km.
2. El código fue completamente documentado y estructurado en módulos, lo cual facilita su comprensión y mantenimiento.
3. Las **visualizaciones generadas** ayudaron a interpretar el proceso evolutivo y la calidad de la solución.
4. El **modelo del problema** se representó correctamente mediante una matriz de distancias completa derivada del grafo inicial.
5. Se comprobó que los **algoritmos evolutivos** son una herramienta poderosa para resolver problemas NP-completos de manera aproximada en tiempos reducidos.



7. Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
- Holland, J. H. (1992). *Adaptation in Natural and Artificial Systems*. MIT Press.
- Michalewicz, Z. (1996). *Genetic Algorithms + Data Structures = Evolution Programs*. Springer.
- Haupt, R. L., & Haupt, S. E. (2004). *Practical Genetic Algorithms*. Wiley.
- Stützle, T., & Dorigo, M. (2004). *Ant Colony Optimization*. MIT Press.